JOUR 2

L'infrastructure décrite en code

Mardi 20 octobre 2026 - 9h10-17h10 - Infrastructure, orchestration et automatisation

Mastère Data Engineering & IA, M2 - EFREI Bordeaux - Alexandre STOPPINI

AU PROGRAMME

- Créer le cluster Kubernetes avec OpenTofu : une commande, autant de fois qu'on veut
- Un environnement dev qui naît fermé : quota, réseau, sécurité des pods, secrets générés
- Le même socle chez un fournisseur de cloud : ce qui change, ce qui reste

JOUR 2

Plan de la journée

390 minutes utiles - deux modules, une lecture cloud, deux ateliers

Créneau	Bloc	Contenu
09h10 - 09h30	QCM	Consolidation de la journée 1, retour sur la CI
09h30 - 10h45	Module 3	OpenTofu, du code à un cluster
10h45 - 11h00	Pause	
11h00 - 12h35	Atelier 3	Le cluster kind et les outils, en code
12h35 - 12h40	Point de midi	Qui n'a pas deux nodes Ready ?
12h40 - 13h40	Déjeuner	
13h40 - 14h25	Module 4	Un environnement, ses garde-fous, ses secrets
14h25 - 14h35	Lecture	Le même socle chez un fournisseur de cloud
14h35 - 14h50	Exercice	Expliquer un plan sans terme technique
14h50 - 15h05	Pause	
15h05 - 16h55	Atelier 4	L'environnement dev
16h55 - 17h10	Bilan	La preuve du jour, ce qui arrive demain

JOUR 2

Le dépôt et les outils, puis le QCM

Dès votre arrivée : Commandes_J2, partie 1 - puis 12 minutes de QCM, non noté

Dès votre arrivée, dans l'ordre

- Votre dossier de projet dans $DEPOT (Commandes_J2, étape 1)
- Tout de suite, dans un second terminal : docker pull de l'image du node kind, 1 Go (étape 2)
- git pull --no-rebase --no-edit cours j2, puis git push
- docker compose down, depuis app : le cluster veut le port 8443
- installer-outils.ps1, puis un nouveau terminal

Puis le QCM et la CI

- Lien du QCM sur Moodle ; seul, sans document
- Après l'envoi : "Afficher le score", puis lisez les justifications
- Onglet Actions de votre dépôt : la dernière CI est-elle verte ?
- Rouge sur le scan : une faille fixable, lisez le rapport

MODULE 3

OpenTofu, du code à un cluster

- Les mots : provider, ressource, variable, sortie, state, plan
- Le cycle init, plan, apply, et la preuve qu'il ne reste rien à faire
- Le langage HCL, le provider kind, les trois couches du dépôt

MODULE 3

Les mots d'abord

Neuf mots qui reviennent toute la semaine

Mot	Ce que c'est
IaC	Décrire dans des fichiers versionnés ce qui doit exister ; un outil le crée
OpenTofu	L'outil du module : version libre de Terraform, langage HCL, commande tofu
Provider	Un plugin qui parle à un système : tehcyx/kind, hashicorp/kubernetes
Resource	Un objet créé et géré : resource "kind_cluster" "velocite"
Variable	Une entrée du code, typée, avec un défaut : var.nb_workers
Output	Une valeur rendue par la couche : tofu output
State	Le fichier où OpenTofu note ce qu'il a créé
Plan	Les changements qu'OpenTofu ferait : + créer, ~ modifier, - détruire
Lock file	.terraform.lock.hcl : versions et hashes des providers

MODULE 3

Pourquoi pas une commande à la main

kind create cluster marche aussi - jusqu'au jour où il faut le refaire

À la main

- La commande exacte vit dans l'historique d'un poste
- Personne ne relit un changement avant qu'il arrive
- Rien ne dit si le cluster d'aujourd'hui est celui d'hier
- Le refaire, c'est se souvenir

Décrit en code

- La description vit dans le dépôt, versionnée
- Un changement passe par une pull request, relue
- Le plan dit l'écart entre le voulu et le réel
- Le refaire, c'est relancer la même commande

MODULE 3

Le cycle : init, plan, apply

Six commandes, et la dernière ligne qui prouve tout

Windows, macOS, Linux - dans le dossier d'une couche

```
tofu init     # télécharge les providers, vérifie
              # leurs hashes (lock file)
tofu plan     # compare code et state : ce qui
              # changerait. Ne touche à rien.
tofu apply    # refait le plan, demande "yes",
              # applique, met le state à jour
tofu plan     # juste après : "No changes."
              # C'est la preuve d'idempotence.
tofu output   # affiche les sorties
tofu destroy  # détruit ce que le state connaît
```

Ce qui casse

Modifier à la main un objet créé par OpenTofu.

Le plan suivant le voit : c'est la dérive. Il propose de remettre l'objet comme le code le décrit.

Le code reste la référence.

MODULE 3

Lire un plan

Avant de taper yes : quoi, combien, rien de détruit sans le vouloir

Sortie de tofu plan - couche 00

```
  # kind_cluster.velocite will be created
  + resource "kind_cluster" "velocite" {
      + name       = "velocite"
      + endpoint   = (known after apply)
      + node_image = "mirror.gcr.io/kindest/node:v1.36.4@sha256:..."
      ...
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

Signe	Ce qui va se passer
`+`	créer
`~`	modifier sur place
`-`	détruire
`-/+`	détruire puis recréer
`(known after apply)`	connu après création

-/+ sur un cluster : tout ce qu'il contient est perdu.

MODULE 3

Le langage HCL, l'essentiel

Une ressource, ses arguments, ses blocs imbriqués ; une variable ; une sortie

HCL - un exemple neutre

```
variable "nom" {
  type    = string
  default = "web"
}

resource "exemple_serveur" "web" {
  name  = var.nom        # une variable
  ready = true
  disque {               # un bloc imbriqué
    taille_go = 20
  }
}

output "adresse" {
  value = exemple_serveur.web.adresse
}
```

À lire dans cet ordre

- resource "type" "nom" : le type vient du provider, le nom est à vous
- argument = valeur : ce que vous fixez
- bloc { } : une partie de l'objet, parfois répétée
- <type>.<nom>.<attribut> : lire une valeur après création
- Les attributs lisibles : documentation du provider, rubrique Read-Only

MODULE 3

Répéter un bloc : dynamic et range

Autant de blocs qu'une liste a d'éléments, sans copier-coller

HCL - un exemple neutre

```
variable "ports" {
  type    = list(object({ interne = number, externe = number }))
  default = [{ interne = 80,  externe = 8080 },
             { interne = 443, externe = 8443 }]
}

resource "exemple_serveur" "web" {
  dynamic "publication" {        # le bloc à répéter
    for_each = var.ports          # une liste, ou range(n)
    content {
      port_interne = publication.value.interne
      port_externe = publication.value.externe
    }
  }
}
```

Trois règles

- for_each prend une liste, jamais un nombre
- range(3) donne [0, 1, 2] : trois copies
- Dans content, l'élément courant s'appelle <nom du bloc>.value

MODULE 3

Le provider kind

La configuration de kind, en YAML, se réécrit en HCL dans un bloc kind_config

Sans OpenTofu : cluster.yaml

```yaml
# kind create cluster --config cluster.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  extraPortMappings:
  - containerPort: 30080
    hostPort: 8090
    listenAddress: "127.0.0.1"
- role: worker
```

Avec OpenTofu : la même chose en HCL

```
resource "kind_cluster" "exemple" {
  name           = "exemple"
  node_image     = "mirror.gcr.io/kindest/node:v1.36.4@sha256:..."
  wait_for_ready = true
  kind_config {
    kind        = "Cluster"
    api_version = "kind.x-k8s.io/v1alpha4"
    node {
      role = "control-plane"
      extra_port_mappings {
        container_port = 30080
        host_port      = 8090
        listen_address = "127.0.0.1"
      }
    }
    node { role = "worker" }
  }
}
```

Les clés en camelCase passent en snake_case ; chaque "- role:" de la liste YAML devient un bloc node { } ; node_image fixe la version de Kubernetes, pinnée par digest ; 127.0.0.1 garde le port sur le poste.

MODULE 3

Trois couches, et pourquoi

Une couche = un dossier = un state ; on sépare ce qui change à des rythmes différents

Couche	Dossier	Crée	Change
00	`infra/00-cluster`	le cluster kind	rarement ; la détruire détruit tout
10	`infra/10-outils`	Argo, Prometheus, Grafana, Loki (fourni)	quelques fois par an
20	`infra/20-env`	un environnement : namespace, garde-fous, secrets	à chaque environnement

Ce que ça donne dans le cluster

```
cluster kind velocite                 couche 00, un seul
|-- argo, observabilite               couche 10, partagée
|     Argo, Prometheus, Grafana, Loki
|-- velocite-dev                      couche 20, workspace dev
|     quota, réseau fermé, Pod Security, Secrets
`-- velocite-prod                     couche 20, workspace prod
      le même code, d'autres valeurs (mercredi)
```

La raison

Une erreur sur un environnement ne doit jamais pouvoir détruire le cluster.

Construire : 00, 10, 20.

Détruire : 20, 10, 00.

MODULE 3

Kubernetes en cinq mots

De quoi lire ce que la couche 00 crée, sans plus pour aujourd'hui

Mot	Ce que c'est
Cluster	Des machines (ici des conteneurs) gérées comme une seule
Node	Une machine ; le control-plane pilote, les workers exécutent
Pod	La plus petite unité lancée : un ou plusieurs conteneurs
Namespace	Un espace isolé du cluster ; dev et prod en auront un
kubectl	Le client en ligne de commande

Windows, macOS, Linux

```
kubectl config current-context
# kind-velocite
kubectl get nodes
kubectl get pods -A
```

MODULE 3

La dérive, en direct

Ce qui se passe quand quelqu'un touche à la main ce que le code décrit

Démonstration - projet demo-iac, étape 3

```
cd demo-iac/03-appli
tofu plan                       # No changes.
kubectl -n demo scale deployment vitrine --replicas=0
tofu plan
# ~ resource "kubernetes_deployment_v1"
#     ~ replicas = "0" -> "2"
# Plan: 0 to add, 1 to change, 0 to destroy.
tofu apply                      # le code gagne
```

Ce qu'on retient

- Le plan voit l'écart entre le code et le réel
- apply remet le réel comme le code
- Une correction à la main se perd au prochain apply
- Corriger, c'est corriger le code

ATELIER 3

Atelier 3 - Le cluster et les outils, en code

95 min - en binôme - pièces : infra/00-cluster/main.tf, outputs.tf, début de DEPLOIEMENT.md

Consignes

- Dans infra/00-cluster/main.tf, écrivez la ressource kind_cluster du A FAIRE, points 1 à 5
- Dans outputs.tf, les trois sorties demandées
- tofu init, tofu plan : combien de ressources ? Lisez avant d'appliquer
- tofu apply, puis kubectl get nodes : deux nodes Ready
- Un second tofu plan doit répondre No changes.
- Appliquez infra/10-outils, fournie complète (phrase du state d'abord)
- DEPLOIEMENT.md, sections 1 à 4 : ce que vous avez tapé, ce qu'on doit voir

Windows ; macOS, Linux : les lignes # Mac/Linux

```
docker info --format "{{.CgroupVersion}}"   # 2
(Get-Command kubectl).Source    # velocite-outils
# Mac/Linux : command -v kubectl
cd "$DEPOT/infra/00-cluster"
tofu init
tofu plan
tofu apply
kubectl get nodes
tofu plan                       # No changes.
$env:TF_VAR_phrase_etat = Read-Host "Phrase"
# Mac/Linux : read -rs TF_VAR_phrase_etat
#             export TF_VAR_phrase_etat
cd "$DEPOT/infra/10-outils"
tofu init
tofu apply                      # 2 à 3 minutes
```

ATELIER 3

Si ça coince

Les blocages relevés pendant la répétition, et leur sortie

Ce qu'on voit	La cause	La sortie
Unsupported argument "for_each" … number	for_each sur un nombre	for_each = range(var.nb_workers)
kubeadm : quarante lignes d'erreur	Docker en cgroup v1	mettre à jour Docker Desktop, puis .wslconfig
kubectl version : client 1.25	le kubectl de Docker Desktop passe avant	nouveau terminal ; PATH : velocite-outils d'abord
Plan : 1 to destroy, 1 to add	un argument qui force la recréation a changé	relire le plan ; ne pas taper yes
La phrase est redemandée au clavier	nouveau terminal	redéfinir TF_VAR_phrase_etat
velocite-config dans git status	le kubeconfig écrit par kind	il est ignoré par .gitignore : vérifier la ligne
port 8443 … already allocated	VéloCité tourne encore en Compose	docker compose down, depuis app

JOUR 2

Point de midi

5 minutes - personne ne part déjeuner sans un cluster qui répond

Windows, macOS, Linux - la preuve

```
cd "$DEPOT"
kubectl get nodes
tofu -chdir=infra/00-cluster plan
# No changes.
```

La preuve du matin, en une phrase

"Notre cluster se recrée à l'identique en une commande, et un second passage ne change rien."

Dites-la à votre binôme, puis sans le mot cluster.

MODULE 4

Un environnement, ses garde-fous, ses secrets

- Un code, deux environnements : modules, workspaces, fichiers de variables
- Un namespace qui naît fermé : quota, limites, réseau, sécurité des pods
- Des secrets générés, jamais écrits, dans un state chiffré

MODULE 4

Les mots d'abord

Huit mots pour la couche 20

Mot	Ce que c'est
Module	Un dossier de code OpenTofu appelé depuis un autre
Workspace	Plusieurs states pour le même code : un par environnement
Fichier tfvars	Les valeurs d'un environnement : envs/dev.tfvars
ResourceQuota	Le plafond de ce que les pods d'un namespace demandent ensemble
LimitRange	Les valeurs par défaut d'un conteneur qui ne déclare rien
NetworkPolicy	Une règle de flux entre pods ; sans règle, tout passe
Pod Security Admission	Un contrôle à l'entrée du namespace qui refuse les pods trop privilégiés
Secret	Un objet qui porte des valeurs sensibles, montées en fichiers

MODULE 4

Un code, deux environnements

Le workspace choisit le state ; le fichier de variables choisit les valeurs

infra/20-env

```
main.tf             le code commun
envs/dev.tfvars     les valeurs de dev
envs/prod.tfvars    les valeurs de prod
modules/espace      namespace, garde-fous
modules/secrets     tous les secrets

tofu workspace select -or-create dev
tofu apply -var-file envs/dev.tfvars
```

HCL - le garde-fou, dans main.tf

```
resource "terraform_data" "garde_workspace" {
  input = terraform.workspace
  lifecycle {
    precondition {
      condition = terraform.workspace == var.env
      error_message = "Workspace ${terraform.workspace}
        mais env = ${var.env}."
    }
  }
}
```

MODULE 4

Le state contient les secrets : il est chiffré

Une phrase de passe, jamais écrite dans un fichier

La phrase, puis vérifier un Secret

```
# Windows
$env:TF_VAR_phrase_etat = Read-Host "Phrase"
# macOS, Linux
read -rs TF_VAR_phrase_etat
export TF_VAR_phrase_etat
# TF_VAR_<nom> devient la variable <nom>

# Partout : les NOMS des clés et leur taille
kubectl -n velocite-dev describe secret velocite-db

# Jamais : -o jsonpath="{.data}"
# base64 n'est pas un chiffrement
```

Trois choses à savoir

- Les mots de passe naissent dans OpenTofu, donc dans le state
- Sans la phrase, plus aucune commande tofu sur la couche : decryption failed ; ce qui tourne continue
- Perdre la phrase : on perd le state, pas le cluster ; tout se recrée, avec de nouveaux secrets
- Un nouveau terminal ne connaît pas la phrase

MODULE 4

Plafond et valeurs par défaut

Le quota compte ; le LimitRange donne de quoi compter

HCL - exemple neutre

```
resource "kubernetes_limit_range_v1" "defauts" {
  metadata {
    name      = "defauts"
    namespace = "exemple"
  }
  spec {
    limit {
      type = "Container"
      default = {           # la limite
        cpu = "500m", memory = "256Mi" }
      default_request = {   # la réservation
        cpu = "50m",  memory = "64Mi" }
    }
  }
}
```

Ce qui casse avec un quota seul

Un pod qui ne déclare aucune ressource est refusé : le quota ne sait pas le compter.

Le LimitRange lui en donne par défaut. 500m = un demi-processeur.

default et default_request sont tous deux dans le LimitRange : la limite et la réservation. Le quota est un autre objet ; il additionne les requests et les limits de tous les pods.

MODULE 4

Un réseau fermé par défaut

Tout refuser, puis autoriser ce qui est nécessaire : d'abord le DNS

HCL - une sortie autorisée, exemple neutre

```
spec {
  pod_selector {}                # 1
  policy_types = ["Egress"]      # 2
  egress {                       # 3
    to {                         # 4 : les DEUX conditions
      namespace_selector {
        match_labels = { "kubernetes.io/metadata.name" = "partage" } }
      pod_selector {
        match_labels = { "app" = "cache" } }
    }
    ports { port = "6379", protocol = "TCP" }   # 5
  }
}
```

L'exemple, ligne par ligne

- 1 pod_selector {} vide : tous les pods du namespace
- 2 Egress : la règle ne parle que de ce qui sort
- 3 egress : une sortie autorisée, le reste fermé
- 4 un seul to : namespace partage ET pods app=cache
- 5 vers le port 6379 en TCP seulement

À L'ATELIER 4, VOS DEUX RÈGLES

tout-refuser : la ligne 1, Ingress et Egress à la ligne 2, et aucun bloc egress ni ingress : tout est fermé.
autoriser-dns : la forme de l'exemple, vers kube-system et les pods k8s-app=kube-dns, port 53 en UDP et en TCP.

MODULE 4

Sécurité des pods, et secrets générés

Le durcissement de Sécurité 1, imposé par le cluster ; des secrets jamais écrits

Labels du namespace

```
pod-security.kubernetes.io/enforce = restricted
pod-security.kubernetes.io/warn    = restricted
```

- restricted refuse : root, capacités Linux, escalade de privilèges, dossier de l'hôte
- enforce refuse ; warn prévient

Ce que merge produit : des tables, pas des valeurs

```
for    -> { a = "Xk3...", b = "p9Q..." }
urls   -> { url = "postgresql://...p9Q..." }
merge  -> { a = "Xk3...", b = "p9Q...",
            url = "postgresql://...p9Q..." }
```

HCL - exemple neutre

```
resource "random_password" "mdp" {
  for_each = toset(["a", "b"])
  length   = 32
}
resource "kubernetes_secret_v1" "ex" {
  metadata { name = "exemple" }
  data = merge(
    { for k in ["a", "b"] :
        k => random_password.mdp[k].result },
    local.urls,
  )
}
```

LECTURE

Le même socle chez un fournisseur de cloud

Réseau, calcul, stockage : ce que kind cache dans Docker, un cloud le loue et le facture

Syllabus	Sur le poste (kind)	Chez un cloud (AWS, par exemple)
Réseau	réseau Docker, ports sur 127.0.0.1	VPC (réseau privé), subnet, Internet gateway, security group ouvert à UNE adresse
Calcul	deux conteneurs Docker	une machine virtuelle louée à l'heure (EC2), avec k3s
Stockage	volumes Docker des nodes	un disque chiffré (EBS) ; le state dans un bucket partagé (S3), avec un verrou
Accès	votre compte sur le poste	une identité par binôme, aux droits limités (IAM)
Coût	rien	à la minute : une machine oubliée se paie

Seule la couche 00 change. Les couches 10 et 20, les manifests, les garde-fous : identiques.

EXERCICE

Expliquer un plan à quelqu'un qui ne fait pas d'informatique

15 minutes - la compétence qui porte le plus de points le 30 novembre

La trame, une phrase chacune

- Ce qui va se passer, en mots de tous les jours
- Ce que ça protège, pour l'usager ou l'entreprise
- Ce qui pourrait mal tourner, et comment on le saurait

Exemple, un plan qui crée un quota

Trop technique : "On ajoute un kubernetes_resource_quota_v1 avec limits.memory à 4Gi."

Réussi : "On fixe un plafond à ce que l'environnement de test peut consommer. Une erreur dans les tests ne peut pas ralentir la production. Si le plafond est trop bas, les nouvelles versions refuseront de démarrer : on le verra tout de suite."

ATELIER 4

Atelier 4 - L'environnement dev

110 min - en binôme - quatre étapes, dans l'ordre ; les commandes sont dans Commandes_J2

Quand	Le fichier	Ce que vous écrivez	Fini quand
15h05 35 min	infra/20-env/ modules/espace/main.tf	Dans labels du namespace : les 2 lignes Pod Security (slide 24). Sous le A FAIRE, 4 resources : le quota plafonds (valeurs de var.quota), le LimitRange defauts (slide 22), les NetworkPolicy tout_refuser et dns (slide 23)	tofu validate répond Success! après chaque resource
15h40 30 min	infra/20-env/ modules/secrets/main.tf	Les 2 Secrets des deux A FAIRE : velocite-db (9 clés), velocite-app (8 clés), exactement les noms listés. Modèle : le Secret velocite-garage, déjà écrit dans le fichier	tofu validate répond Success!
16h10 20 min	le terminal, dans infra/20-env	Workspace dev, puis plan : lisez le nombre avant d'appliquer (45 to add). Apply, puis les vérifications de Commandes_J2	le pod essai est refusé ; un second plan répond No changes
16h30 25 min	docs/DEPLOIEMENT.md docs/PARAMETRES.md scripts/up.ps1	DEPLOIEMENT, section 4 : les commandes qui ont marché, et ce qu'on doit voir. PARAMETRES : les tableaux Couche 20 et Les secrets. up.ps1 : les 2 premiers A FAIRE, sur le modèle de l'étape Cluster	up.ps1 -Env dev -SansImages, relancé, ne change rien

Avant l'étape 3 : la phrase définie dans le terminal (slide 21). **Bloqué** : la slide 28, Si ça coince, dans le PDF du cours sur Moodle.

ATELIER 4

Si ça coince

Les blocages relevés pendant la répétition, et leur sortie

Ce qu'on voit	La cause	La sortie
Workspace dev mais env = prod	le mauvais fichier de variables	changer de fichier ou de workspace
Le pod essai démarre	labels Pod Security absents ou faux	relire la slide 24, deux clés
Le plan propose de recréer un Secret	une clé a changé de nom	comparer avec la liste du A FAIRE
Error acquiring the state lock … decryption failed	mauvaise phrase, pas un verrou	retaper la phrase ; rien n'est perdu
Le state s'affiche sur une seule ligne immense	le state chiffré	c'est normal : il est illisible, c'est le but

BILAN

La preuve du jour

Trois commandes, depuis $DEPOT : deux No changes, puis un refus

La commande	Ce qu'elle prouve	La réponse attendue
`tofu -chdir=infra/00-cluster plan`	Le cluster est exactement celui du code : rien changé à la main, rien d'oublié	No changes.
`tofu -chdir=infra/20-env plan -var-file envs/dev.tfvars`	L'environnement dev aussi, garde-fous et secrets compris (phrase définie)	No changes.
`kubectl -n velocite-dev run essai --image=mirror.gcr.io/library/busybox:1.37 --restart=Never -- id`	Le namespace refuse un programme qui tourne en root : le cluster impose la sécurité	Error from server (Forbidden) ... restricted

En une phrase, pour la directrice financière

"Notre environnement de test se recrée à l'identique en une commande et ne peut pas consommer plus que ce qu'on lui accorde. Il refuse tout programme qui demanderait trop de droits."

BILAN

Demain : VéloCité sur Kubernetes

Mercredi 21 octobre - déployer, puis une mise à jour qui casse

Ce qui arrive

- Les manifestes des services, avec Kustomize
- Les NetworkPolicy des applications
- La prod, puis une mise à jour qui casse, et le retour arrière

Ce soir, cinq minutes

- Laisser le cluster en marche, ou le détruire : up.ps1 le recrée
- Commit et push de votre travail
- Noter votre phrase de passe là où vous la retrouverez, hors du dépôt